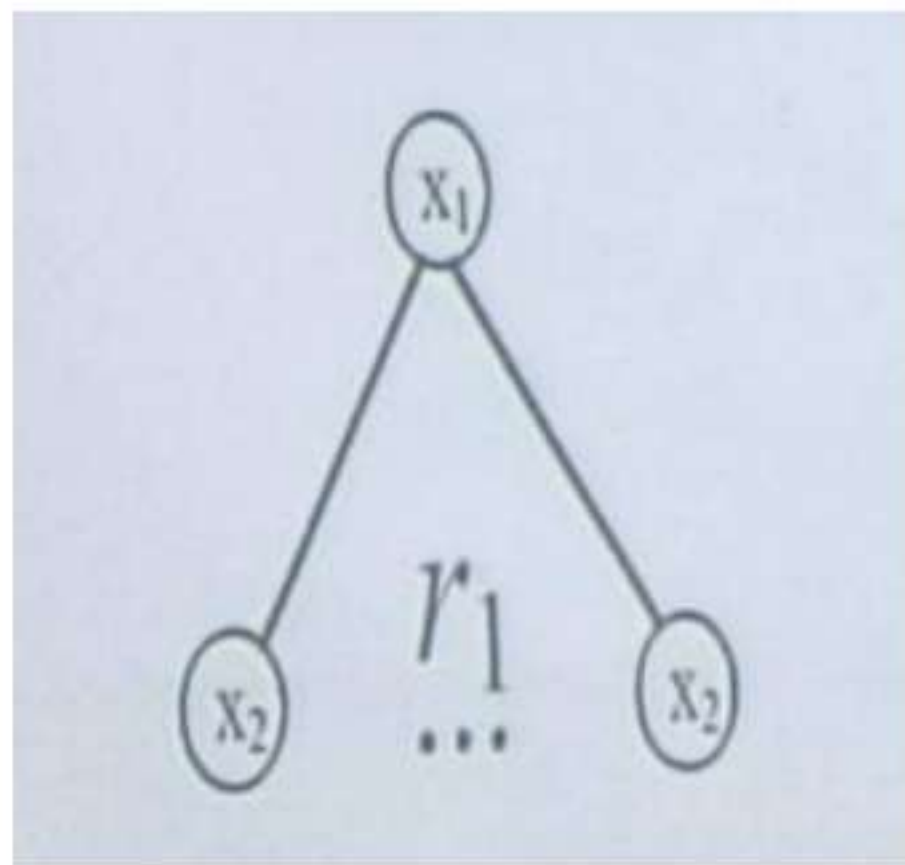
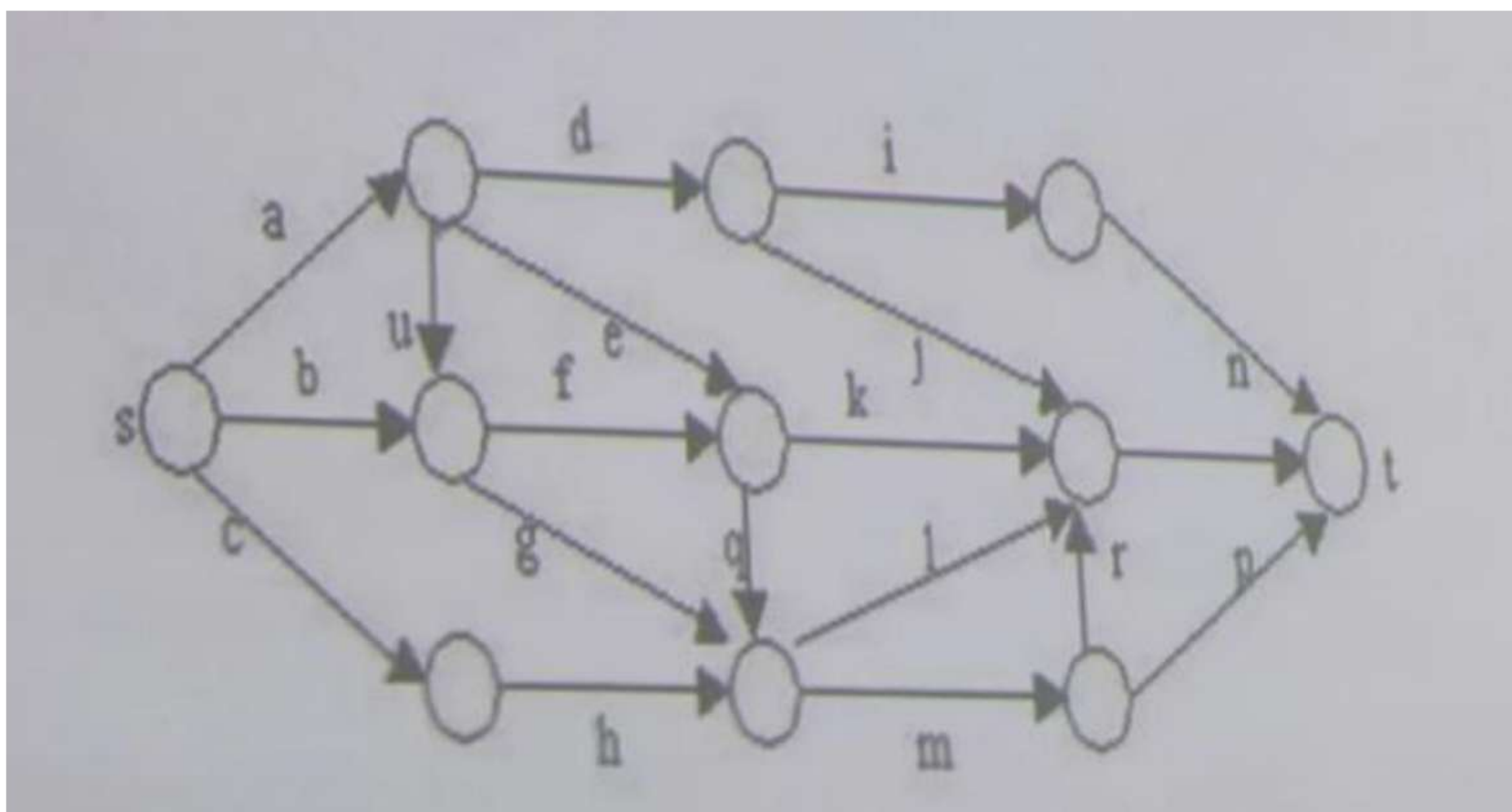


分支限界的设计思路

设求解最大化问题，解向量为 $\mathbf{X}=(x_1,\dots,x_n)$ ， x_i 的取值范围为 S_i ， $|S_i|=r_i$ 。在使用分支限界搜索问题的解空间树时，先根据限界函数估算目标函数的界[down,up]，然后从根结点出发，扩展根结点的 r_1 个孩子结点，从而构成分量 x_1 的 r_1 种可能的取值方式。



下面以一个例子来说明单源最短路径问题：在下图所给的有向图 G 中，每一边都有一个非负边权。要求图 G 的从源顶点 s 到目标顶点 t 之间的最短路径。



总结： 从 0-1 背包问题的搜索过程可看出：与回溯法相比，分支限界法可根据限界函数不断调整搜索方向，选择最可能得到最优解的子树优先进行搜索，从而找到问题的解。

分治法不同的是，适合于用动态规划求解的问题是（**A**）。

- A. 经分解得到子问题往往不是互相独立的
- B. 经分解得到子问题往往是互相独立的
- C. 经分解得到子问题往往是互相交叉的
- D. 经分解得到子问题往往是任意的

1. 动态规划与分治法的关键区别： - 分治法：子问题相互独立，不存在重叠 - 动态规划：子问题之间存在重叠，不是互相独立的

2. 为什么选 **A**： - 动态规划的核心特征就是处理重叠子问题 - 通过存储子问题的解避免重复计算 - 正是因为子问题不独立，才需要记忆化

3. 排除其他选项： **B** 错误：描述的是分治法的特征 **C** 错误：'交叉'说法不准确，应该是'重叠' **D** 错误：子问题不是任意的，而是有明确的重叠关系

左倾红黑树(Left-Leaning Red-Black Tree, LLRB)与 2-3 树之间有着密切的关系,下列关于两者关系的描述中，正确的是（**B**）。

A.左倾红黑树是对 2-3 树的完全二叉树实现,所有节点的结构与 2-3 树完全一致

B.2-3 树可以通过将 3 节点拆分成一个红黑树中的红链接来等价转换为左倾红黑树。

C.左倾红黑树无法保持 2-3 树的平衡性，因为红黑树的插入方式不同。

D.左倾红黑树是一种二叉查找树,与 2-3 树在性能和逻辑上没有关联。

1. LLRB 和 2-3 树的关系：

- LLRB 是 2-3 树的一种二叉树表示
- 通过红链接将 2-3 树的 3 节点模拟表示
- 保持了 2-3 树的完美平衡性质

2. 选 B 的理由：

- 2-3 树的 3 节点(包含 2 个键)可以通过红链接拆分表示
- 红链接总是左倾的，保证了树的唯一性
- 这种转换是完全等价的，维持了相同的平衡性质

3. 其他选项错误原因：

A 错误：不是完全二叉树实现，而是通过红链接模拟

C 错误：LLRB 确实保持了 2-3 树的完美平衡性

D 错误：两者有着密切的逻辑关系，LLRB 就是 2-3 树的一种实现方式

4. 关键概念：

- 红链接：表示 2-3 树中 3 节点的两个键之间的连接
- 左倾：确保红链接只出现在左边，保证表示的唯一性
- 平衡性：通过红黑规则维持 2-3 树的平衡特性

在使用哈希表(Hash Table)存储数据时,下列关于哈希函数和冲突处理的描述中,正确的是(B)。

A. 哈希函数的主要目的是保证哈希表中的所有键值都均匀分布在存储空间中,从而完全避免冲突的发生。

B. 拉链法(Chaining)通过在每个哈希表槽位存储一个链表来解决冲突,但链表会导致查找操作的时间复杂度变为 $O(n)$ 。

C. 线性探测(Linear Probing)是一种开放地址法,其核心思想是通过哈希函数的二次更新来减少冲突概率。

D. 哈希函数设计得越复杂,哈希表的性能越高,因为复杂的函数能更好地避免冲突。

1. 拉链法(Chaining)的特点:

- 每个槽位维护一个链表存储冲突的元素
- 最坏情况下所有元素都在同一链表中,此时查找复杂度为 $O(n)$
- 但平均情况下,如果哈希函数足够好,复杂度接近 $O(1)$

2. 其他选项错误原因:

A 错误:

- 哈希函数不可能完全避免冲突
- 只能尽量减少冲突概率

C 错误:

- 线性探测是通过线性步长寻找下一个空槽

- 不是通过二次更新哈希函数

D 错误：

- 哈希函数应该简单高效
- 复杂度和冲突避免不是正相关

3. 哈希表的关键设计原则：

- 哈希函数要快速计算
- 尽量保证均匀分布
- 合理处理冲突情况
- 在时间和空间上取得平衡

关于 2-3 树的性质和操作，下列描述中，正确的是（C）。

A.2-3 树是一种二叉树，其中每个节点最多有两个子节点，且所有叶子节点深度相同。

B.2-3 树的插入操作通过将 2 节点转化为 3 节点完成，且不需要额外的节点分裂操作。

C.2-3 树的平衡性是通过节点分裂和父节点的合并维护的，因此插入和删除的时间复杂度始终为 $O(\log n)$ 。

D.在 2-3 树中，删除元素时，可能需要重构整棵树以保持平衡性，从而导致最坏时间复杂度为 $O(n)$ 。

1. 2-3 树的核心特性： - 完美平衡：所有叶节点到根节点距离相同 -

节点类型:2-节点(1个键,2个子节点)或3-节点(2个键,3个子节点) - 自底向上生长: 通过分裂和合并保持平衡

2. 为什么选 C: - 平衡维护机制: * 插入时可能触发节点分裂 * 分裂后将中间键上移到父节点 * 删除时可能需要借节点或合并 - 时间复杂度: * 树高度始终为 $O(\log n)$ * 分裂/合并操作是常数时间 * 因此总操作时间保持在 $O(\log n)$

3. 其他选项错误原因: A 错误: - 2-3 树不是二叉树 - 节点可以有 3 个子节点 B 错误: - 插入可能需要节点分裂 - 3 节点插入时必定分裂 D 错误: - 删除操作不需要重构整棵树 - 只需局部调整, 复杂度仍是 $O(\log n)$

4. 关键操作特点: - 插入: 自底向上分裂 - 删除: 借节点或合并 - 查找: 与 B 树类似 - 所有操作都保持平衡性

关于并查集 (Union-Find) 的描述, 下列选项中, 正确的是 (A)。

A. 并查集主要用于解决动态连通性问题, 可以高效地支持插入元素的操作。

B. 在使用路径压缩优化时, 并查集的查找操作时间复杂度会退化为 $O(n)$ 。

C. 按权重合并优化的核心思想是始终将树的高度较小的集合连接到树的高度较大的集合上, 以减少树的深度。

D. 并查集无法动态维护多个独立的连通分量, 每次更新后需要重构整个数据结构以保持正确性。

1. 并查集(Union-Find)的基本特征:

- 主要用途: 解决动态连通性问题
- 核心操作: 查找(Find)和合并(Union)
- 支持动态插入和高效查询
- 近乎常数时间的操作复杂度

2. 选 A 的理由:

- 动态连通性:
 - * 能够动态处理元素间的连接关系
 - * 快速判断两个元素是否连通
- 高效插入:
 - * 新元素可以快速加入
 - * 不需要重构现有结构

3. 其他选项错误分析:

B 错误:

- 路径压缩优化实际上降低了查找复杂度
- 接近 $O(1)$, 而不是退化到 $O(n)$

C 错误:

- 描述的是按秩合并(rank)
- 不是按树高度而是按秩合并

D 错误：

- 并查集天然支持多个连通分量
- 不需要重构数据结构

4. 关键优化技术：

- 路径压缩：
 - * 查找时直接连接到根节点
 - * 显著降低树的高度
- 按秩合并：
 - * 基于树的秩来决定合并方向
 - * 避免树变得过深

1. 简述分治法的基本思想。

将一个规模为 n 的问题分解为 k 个规模较小的子问题，这些子问题互相独立且与原问题相同。递归地解这些子问题，然后将各个子问题的解合并得到原问题的解。

2. 动态规划的基本思想。

动态规划是一种解决最优化问题的算法策略，它将复杂问题分解为相互依赖的子问题，并通过存储这些子问题的解来避免重复计算。与贪

心算法自顶向下做局部选择和分治算法处理独立子问题不同，动态规划的独特之处在于它允许子问题之间存在重叠，通过记忆化的方式保存和复用子问题的解，从而在处理具有重复子问题特征的场景时能够高效地得到全局最优解。

3. 阐述分支限界法的搜索策略。

在扩展结点处，先生成其所有的儿子结点（分支），然后再从当前的活结点表中选择下一个扩展结点。为了有效地选择下一扩展结点，加速搜索的进程，在每一个活结点处，计算一个函数值（限界），并根据函数值，从当前活结点表中选择一个最有利的结点作为扩展结点，使搜索朝着解空间上有最优解的分支推进，以便尽快地找出一个最优解。

4. 简述蒙特卡洛、拉斯维加斯、舍伍德三种概率算法的设计思想与特点。

蒙特卡罗 (Monte Carlo) 算法在一般情况下可以保证对问题的所有实例都以高概率给出正确解，但是通常无法判定一个具体解是否正确。设 p 是一个实数，且 $1/2 < p < 1$ 。如果一个蒙特卡罗算法对于问题的任一实例得到正确解的概率不小于 p ，则称该蒙特卡罗算法是 p 正确的，且称 $p-1/2$ 是该算法的优势。如果对于同一实例，蒙特卡罗算法不会给出 2 个不同的正确解答，则称该蒙特卡罗算法是一致的。

拉斯维加斯 (Las Vegas) 算法的一个显著特征是它所作的随机性决策有可能导致算法找不到所需的解。

```
void obstinate(Object x, Object y) {
```



```

// 反复调用拉斯维加斯算法 LV(x, y), 直到找到问题的一个
解 y

bool success = false;

while (!success) success = LV(x, y);

}

```

设 $p(x)$ 是对输入 x 调用拉斯维加斯算法获得问题的一个解的概率。一个正确的拉斯维加斯算法应该对所有输入 x 均有 $p(x) > 0$ 。

舍伍德算法设 A 是一个确定性算法，当它的输入实例为 x 时所需的计算时间记为 $t_A(x)$ 。设 X_n 是算法 A 的输入规模为 n 的实例的全体，则当问题的输入规模为 n 时，算法 A 所需的平均时间为：
 $t_A(n) = \sum_{x \in X_n} t_A(x) / |X_n|$ ，这显然不能排除存在 $x \in X_n$ ，使得 $t_A(x)$ 很大的可能性。我们希望获得一个概率算法 B ，使得对问题的输入规模为 n 的每一个实例均有： $t_B(n) = t_A(n) + s(n)$

这就是舍伍德算法设计的基本思想，当 $s(n)$ 与 $t_A(n)$ 相比可以忽略时，舍伍德算法可以获得很好的平均性能。

1. 给定已按升序排好序的 n 个元素 $a[0:n-1]$ ，现要在这 n 个元素中找出一特定元素 x 。据此设计二分搜索算法。

```

template<class Type>

int BinarySearch(Type a[], const Type& x, int l, int r) {

    while (l <= r) {

```



```

        int m = (l + (r-l) / 2); //防止整数溢出

        if (x == a[m]) return m;

        if (x < a[m]) r = m - 1; else l = m + 1;

    }

    return -1;

}

// 假设 a[] 是已经排序好的数组，n 是数组的长度，x 是要查找的
// 元素

// 调用 BinarySearch 函数进行查找

int index = BinarySearch(a, x, 0, n - 1);

if (index != -1) {

    // 找到了元素 x

} else {

    // 元素 x 不在数组中

}

```

剩下的代码是关于归并排序（**Merge Sort**）算法的实现。归并排序是一种分治算法，它将数组分成两半，分别对每一半进行排序，然后将排序好的两半合并在一起。这个过程递归地进行，直到整个数组排序完成。下面是归并排序算法的代码整合：

```

template<class Type>

```



```
void Mergesort(Type a[], int left, int right)
{
    if (left < right) {
        int i = (left + right) / 2;  // i = left + (right - left) / 2; 明天如果后面/2 是固定的填前面那个，不是的话填后面那个

        Mergesort(a, left, i);  // 对左半部分进行排序
        Mergesort(a, i + 1, right);  // 对右半部分进行排序
        Merge(a, left, i, right);  // 合并两个已排序的半部分（AI 写的这行）
    }
}
```